

LINGUAGEM DE PROGRAMAÇÃO C++

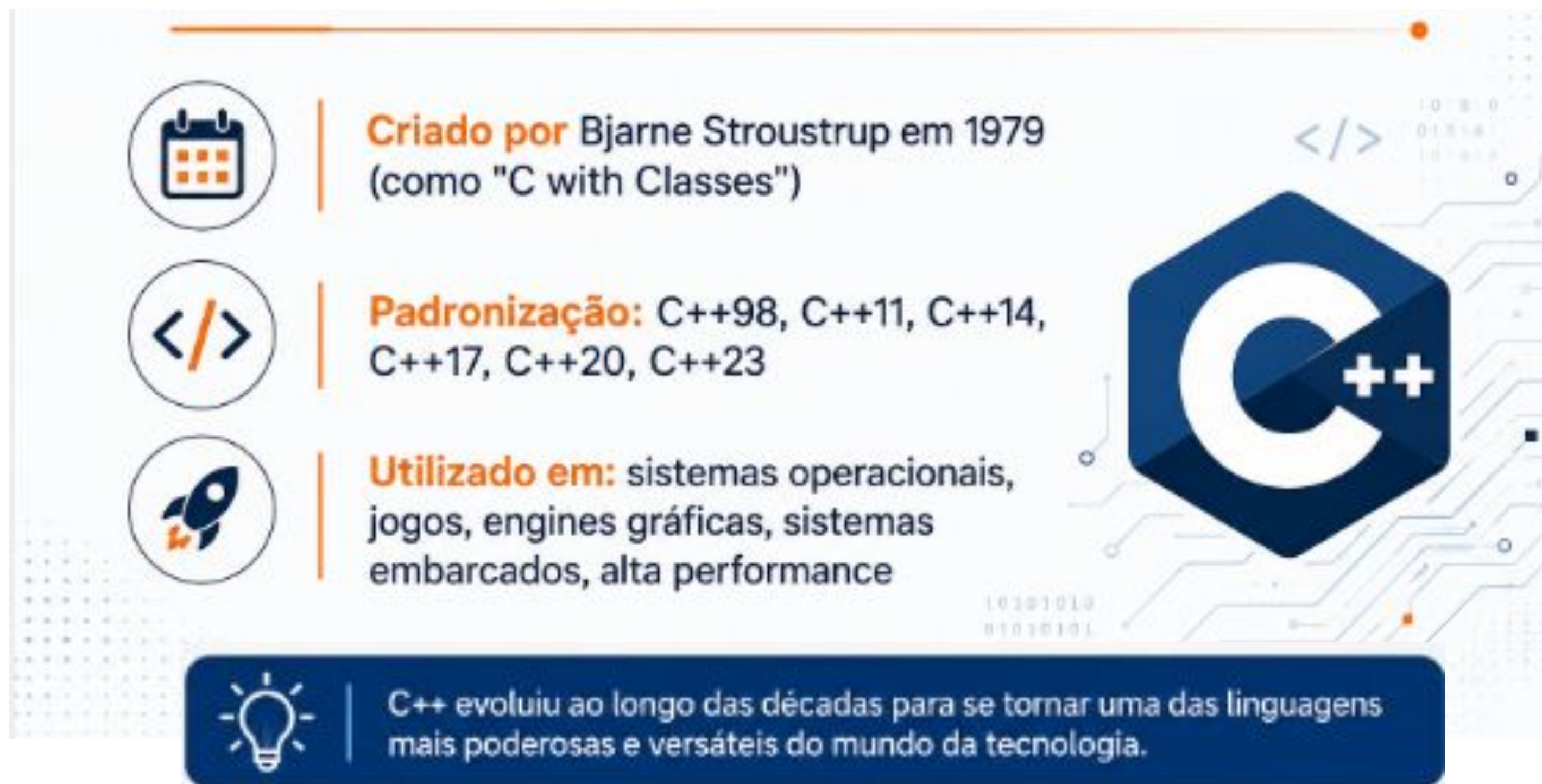
Curso de Capacitação em Desenvolvimento Full Stack - MÓDULO 08 (40H)

Elaborado por: Prof(a). Esp. Érika D. G. R. Dos Santos

INTRODUÇÃO AO C++

AULA 1 : LINGUAGEM DE PROGRAMAÇÃO C++

História e Evolução da Linguagem



The infographic features a light blue background with a subtle circuit pattern. On the left, three circular icons are arranged vertically: a calendar, a code symbol, and a rocket. To the right of these icons, three text blocks provide key information about C++. A large C++ logo is positioned to the right of the middle text block. At the bottom, a dark blue banner contains a lightbulb icon and a concluding statement. The overall design is clean and modern, using a color palette of blue, orange, and white.

- Criado por** Bjarne Stroustrup em 1979 (como "C with Classes")
- Padronização:** C++98, C++11, C++14, C++17, C++20, C++23
- Utilizado em:** sistemas operacionais, jogos, engines gráficas, sistemas embarcados, alta performance

C++ evoluiu ao longo das décadas para se tornar uma das linguagens mais poderosas e versáteis do mundo da tecnologia.

Por que aprender C++? 🤔



- Evoluiu a partir da linguagem C, mantendo sua eficiência e desempenho;
- Poderosa, flexível e abrangente – permite programação de baixo e alto nível;
- Aplicações: sistemas embarcados, automação industrial, jogos, engines gráficas;
- Suporta múltiplos paradigmas: estruturado, POO (classes, herança, polimorfismo) e genérico (templates);
- C++ é amplamente utilizado em áreas que exigem alta performance e controle direto de hardware;

Principais Aplicações

Segurança Kernel Gerenciamento de Recursos
Desempenho **Sistemas Operacionais** Processos
Estabilidade **INFRAESTRUTURA** Memória

Compiladores

Análise Léxica Otimização Geração de Código



Bancos de Dados

Armazenamento Consultas Transações Confiabilidade

Gráficos 3D Física Inteligência Artificial
 **Motores de Jogos**

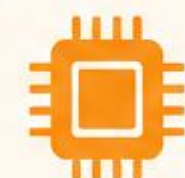
ALTA PERFORMANCE

 **Simulações Científicas**

Modelagem Cálculos Complexos Precisão

 **Sistemas Financeiros**

Baixa Latência Confiabilidade Análise de Risco Transações

Dispositivos Tempo Real Eficiência
 **Sistemas Embarcados**

CONTROLE DIRETO DE HARDWARE

 **Automação Industrial**

CLPs Monitoramento Processos

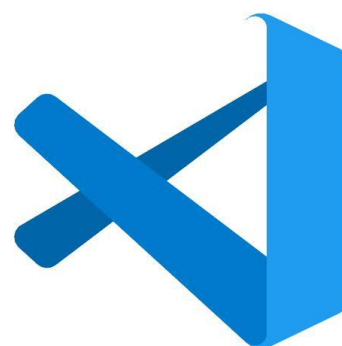
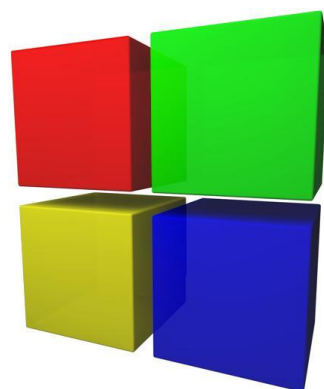
 **Robótica**

Sensores Controle de Motores Navegação

Configuração do Ambiente

IDE e Compilador

- IDEs sugeridas: Code::Blocks, VS Code, Replit (online)
- Compilador: GCC (g++)
- Passos: instalar IDE → configurar compilador → criar projeto → escrever código → compilar → executar



Principais IDEs para Desenvolvimento em C++

1. Code::Blocks



Code::
Blocks

- ✓ Gratuita e leve.
- ✓ Muito utilizada em cursos introdutórios de C e C++.
- ✓ Possui compilador integrado em muitas distribuições.

2. VS Code



Visual Studio
Code

- ✓ Editor moderno e altamente personalizável.
- ✓ Suporte a C++ por extensões.
- ✓ Integração com Git, depuração e terminal.

3. Replit (online)



replit

- ✓ Funciona diretamente no navegador.
- ✓ Não requer instalação.
- ✓ Ideal para aulas, exercícios e colaboração online.

Vamos Praticar?



Primeiro Programa em C++: Hello World

cpp

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    cout << "Hello World!" << endl;
```

```
    return 0;
```

```
}
```

#include <iostream>: biblioteca de entrada/saída

main(): função principal (ponto de entrada)

cout: saída de dados

return 0: indica execução bem-sucedida

Palavras Reservadas do C++

Referência para Consulta (versão moderna)

The diagram illustrates the organization of C++ reserved keywords. It features a central text box with a speech bubble pointing to a list of keywords. The keywords are grouped into five categories, each represented by a colored box with an icon and a label:

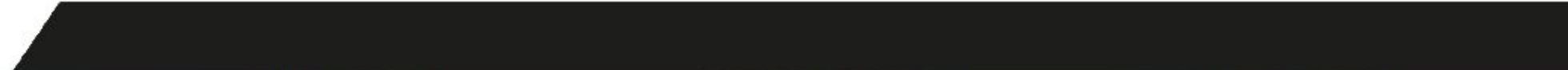
- Tipos** (Types): Represented by a database icon. Keywords shown include `int`, `char`, and `signed`.
- Cor** (Control): Represented by a flowchart icon. Keywords shown include `for`.
- POO** (Object-Oriented Programming): Represented by a cube icon. Keywords shown include `new` and `delete`.
- Memória/Outros** (Memory/Other): Represented by a microchip icon. Keywords shown include `const`, `static`, `extern`, `sizeof`, `typedef`, `enum`, `union`, and `template`.

The central text box contains the following text:

Estas são as palavras reservadas do C++. Não se assustem! Vocês vão aprender cada uma delas no momento certo ao longo das 11 aulas. Este slide está aqui apenas para referência.

CONCEITOS BÁSICOS

AULA 2: LINGUAGEM DE PROGRAMAÇÃO C++



Variáveis e Constantes

Armazenando Dados na Memória

- Variável: espaço na memória com um nome e um tipo
- Declaração: `tipo nome;` (ex: `int idade;`)
- Inicialização: `tipo nome = valor;` (ex: `int idade = 18;`)
- Constante: valor fixo – `const tipo NOME = valor;` (ex: `const double PI = 3.1415;`)



Dica: use nomes significativos para suas variáveis e constantes.



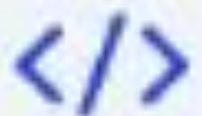
Tipos de Dados Primitivos

Inteiros, Reais, Caracteres e Booleanos

- `int`: números inteiros (ex: 10, -5, 0)
- `float` / `double`: números reais (ex: 3.14, 2.5)
- `char`: um caractere (ex: 'A', '7', '#')
- `bool`: verdadeiro ou falso (`true` / `false`)
- `void`: sem tipo (usado em funções sem retorno)



Esses tipos formam a base para declarar variáveis e manipular dados em C++.



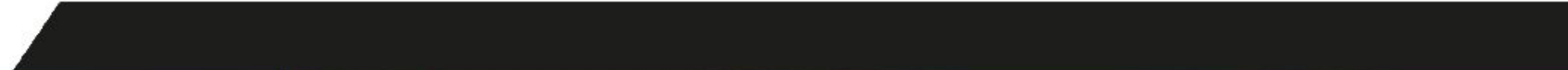
Operadores

Aritméticos, Relacionais e Lógicos

- **Aritméticos:** `+` (soma), `-` (subtração), `*` (multiplicação), `/` (divisão), `%` (módulo)
- **Relacionais:** `==` (igual), `!=` (diferente), `>` (maior), `<` (menor), `>=` (maior ou igual), `<=` (menor ou igual)
- **Lógicos:** `&&` (E), `||` (OU), `!` (NÃO)

ENTRADA E SAÍDA DE DADOS

AULA 3: LINGUAGEM DE PROGRAMAÇÃO C++



Entrada e Saída com cin e cout

Interagindo com o Usuário

- `cout <<` : saída de dados (console)
- `cin >>` : entrada de dados (teclado)
- `endl` : quebra de linha
- Exemplo:

```
cpp
```

```
int idade;
```

```
cout << "Digite sua idade: ";
```

```
cin >> idade;
```

```
cout << "Voce tem " << idade << " anos" << endl;
```

Formatação de Saída

setprecision, fixed e setw

- `#include <iomanip>` (biblioteca necessária)
- `fixed`: notação de ponto fixo
- `setprecision(n)`: define n casas decimais
- `setw(n)`: define largura do campo
- Exemplo:

cpp

```
cout << fixed << setprecision(2);<br>cout << setw(10) << "Nome" <<  
setw(5) << 10;<br>
```

Comentários e Documentação

Boas Práticas de Código

- Comentário de linha: `// isso é um comentário`
- Comentário de bloco: `/* comentário de múltiplas linhas */`
- Boas práticas: nomes significativos, código limpo, documentar funções complexas